这是一个非常典型的**组件化开发**场景。要实现“默认按钮可配置、逻辑可扩展（继承/重写/叠加）”，最轻量且符合 QML 特性的方案是结合 `Repeater` 动态生成按钮，并利用 **JavaScript 函数作为属性（Function Properties）** 来处理逻辑。

通过这种方式，你的 `Bus.js` 仅作为底层的通信管道，而业务逻辑的“重写”则在 QML 层面完成。

0.0.1 1. 设计思路：数据驱动的按钮列表

在 `Head.qml` 中，我们不把按钮写死，而是定义一个列表。每个按钮包含：

- **text**: 显示文字。
- **action**: 默认执行的函数。
- **visible**: 是否显示（用于控制开关）。

0.0.1 2. 核心组件：`Head.qml`

QML

```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts
import "./Bus.js" as Bus

Item {
    id: headRoot
    height: 60

    // 1. 定义默认的业务逻辑方法（可被外部重写）
    property var onSearchAction: function(text) {
        console.log("执行默认搜索逻辑")
        Bus.emit("search_requested", text)
    }

    property var onResetAction: function() {
        console.log("执行默认重置逻辑")
        Bus.emit("reset_requested", {})
    }

    // 2. 按钮配置开关（默认全部开启）
    property bool showSearchBtn: true
    property bool showResetBtn: true

    RowLayout {
        anchors.fill: parent
        anchors.margins: 10
```

```
        spacing: 10

        TextField { id: input; Layout.fillWidth: true; placeholderText: "输入..." }

        // 搜索按钮
        Button {
            text: "搜索"
            visible: headRoot.showSearchBtn
            onClicked: headRoot.onSearchAction(input.text) // 调用属性函数
        }

        // 重置按钮
        Button {
            text: "重置"
            visible: headRoot.showResetBtn
            onClicked: headRoot.onResetAction() // 调用属性函数
        }
    }
}
```

0.0.1 3. 如何实现：继承、重写与叠加

当你使用 `SearchLayoutTemplate` 时，你可以通过以下三种方式灵活控制：

0.0.0.1 A. 隐藏不需要的按钮

QML

```
SearchLayoutTemplate {
    headerContent: Head {
        showResetBtn: false // 仅仅关闭重置按钮
    }
}
```

0.0.0.2 B. 重写方法（完全替换逻辑）

QML

```
SearchLayoutTemplate {
    headerContent: Head {
        // 直接重新赋值函数，完全覆盖默认行为
        onSearchAction: function(text) {
            console.log("这是新的搜索逻辑，不再发送 Bus 消息")
            customDialog.show(text)
        }
    }
}
```

0.0.0.3 C. 叠加方法（保留默认 + 增加新功能）

QML

```
SearchLayoutTemplate {
    headerContent: Head {
        // 保存旧的引用
        property var oldSearch: onSearchAction

        onSearchAction: function(text) {
            console.log("第一步：执行叠加的预处理")
            // 执行原来的默认逻辑
            oldSearch(text)
            console.log("第二步：执行叠加的后续操作")
        }
    }
}
```

0.0.1 4. 配合 `Bus.js` 的解耦架构

你的 `Bus.js` 依然保持纯净，它只负责传递最终的结果：

JavaScript

```
// Bus.js
.pragma library
var _subscribers = {};

function on(msg, cb) { /* ... 之前的代码 ... */ }
function emit(msg, data) { /* ... 之前的代码 ... */ }
```

0.0.1 编写纯 JS 通信中心（`Bus.js`）

JavaScript

```
// Bus.js
.pragma library

// 存储订阅者的回调函数
// key 为消息名称，value 为函数数组
var _subscribers = {};
```

```

/**
 * 订阅消息
 * @param {string} messageName 消息名称
 * @param {var} callback 回调函数
 */
function on(messageName, callback) {
    if (!_subscribers[messageName]) {
        _subscribers[messageName] = [];
    }
    _subscribers[messageName].push(callback);
    console.log("[Bus] 订阅成功:", messageName);
}

/**
 * 发送消息
 * @param {string} messageName 消息名称
 * @param {var} data 传递的数据
 */
function emit(messageName, data) {
    if (_subscribers[messageName]) {
        _subscribers[messageName].forEach(function(callback) {
            callback(data);
        });
        console.log("[Bus] 消息已送达:", messageName);
    }
}
```

0.0.1 2. 在组件中使用（实现 MVVM 与跨组件解耦）

0.0.0.1 Head.qml (发送方)

QML

```
import QtQuick
import "./Bus.js" as Bus
import "./HeadLogic.js" as Logic

Item {
    Button {
        onClicked: {
            let keyword = input.text
            // 逻辑处理
            let finalKey = Logic.format(keyword)
            // 纯 JS 发送，不依赖任何 QML 属性或信号
            Bus.emit("search_requested", finalKey)
        }
    }
```

```
}
```

0.0.0.2 Body.qml (接收方)

QML

```
import QtQuick
import "./Bus.js" as Bus

Item {
    id: bodyRoot

    Component.onCompleted: {
        // 在组件加载时订阅消息
        Bus.on("search_requested", function(data) {
            console.log("Body 收到搜索请求:", data)
            bodyRoot.startLoading(data)
        })
    }

    function startLoading(key) {
        // 执行数据库查询逻辑...
    }
}
```

```
TextField {
    id: inputUser
    objectName: "userName" // 1. 必须定义 ID
    placeholderText: "请输入用户名"

    // 2. 定义协议属性 formValue
    // logic: 读取时取 text，写入时修改 text
    property var formValue: text

    // 3. 响应恢复操作
    // 当模板调用 item.formValue = xxx 时，这个信号被触发
    onFormValueChanged: {
        // 只有当新值和当前 text 不一样时才赋值，避免循环（虽然 QML 内部有保护）
        if (text !== formValue) {
            text = formValue
        }
    }
}
```

```
import QtQuick
import QtQuick.Layouts
import "./FormManager.js" as FormMgr

ColumnLayout {
    id: root

    // ... 其他布局代码 (Header, Body, Footer) ...

    property alias bodyContent: bodyContainer.data
    Item {
        id: bodyContainer
        // ...
    }

    // ==========================================
    // 状态管理功能区
    // ==========================================

    // 内部变量：用于存储暂存的数据快照
    property var _snapshotData: null

    // 功能 1：暂存
    function stash() {
        _snapshotData = FormMgr.saveState(bodyContainer);
        console.log("Body区状态已暂存");
    }

    // 功能 2：恢复
    function restore() {
        if (_snapshotData) {
            FormMgr.restoreState(bodyContainer, _snapshotData);
            console.log("Body区状态已恢复");
        } else {
            console.warn("没有可恢复的暂存数据");
        }
    }

    // 功能 3：重置 (可选)
    // 也可以扩展一个 resetValue 属性来实现清空表单
}
```

```
// FormManager.js
```

```
.pragma library

/**
 * 递归保存状态
 * @param {Item} rootItem - 要遍历的根节点 (如 bodyContainer)
 * @returns {Object} - 包含所有数据的 JSON 对象 { "key": value }
 */
function saveState(rootItem) {
    var data = {};
    _traverseAndSave(rootItem, data);
    return data;
}

/**
 * 递归恢复状态
 * @param {Item} rootItem - 要遍历的根节点
 * @param {Object} savedData - 之前保存的 JSON 数据
 */
function restoreState(rootItem, savedData) {
    if (!savedData) return;
    _traverseAndRestore(rootItem, savedData);
}

// --- 私有辅助函数 ---

function _traverseAndSave(item, data) {
    // 核心契约检查:
    // 1. 必须有 objectName (作为存储的 Key)
    // 2. 必须有 formValue 属性 (作为存储的 Value)
    if (item.objectName && item.hasOwnProperty("formValue")) {
        // 直接读取 formValue，不在乎它是 text 还是 int 还是 object
        data[item.objectName] = item.formValue;

        console.log(`[Save] Key: ${item.objectName}, Value: ${item.formValue}`);
    }

    // 继续递归子节点
    if (item.children) {
        for (var i = 0; i < item.children.length; i++) {
            _traverseAndSave(item.children[i], data);
        }
    }
}

function _traverseAndRestore(item, data) {
    // 核心契约检查
    if (item.objectName && item.hasOwnProperty("formValue")) {
        // 检查数据源里有没有这个 Key
        if (data.hasOwnProperty(item.objectName)) {
```

```
            var val = data[item.objectName];

            // 直接写入 formValue
            // 控件内部的 onFormValueChanged 会负责将这个值同步给 UI
            item.formValue = val;

            console.log(`[Restore] Key: ${item.objectName}, Value: ${val}`);
        }
    }

    // 继续递归子节点
    if (item.children) {
        for (var i = 0; i < item.children.length; i++) {
            _traverseAndRestore(item.children[i], data);
        }
    }
}
```